

Expressões Regulares em Java

Como ler, escrever e entender padrões de texto com Pattern e Matcher

8 capítulos • Pattern, Matcher, grupos e quantificadores • Explicações reescritas do zero

CONTEÚDO DA APOSTILA

- 01** Por que Expressões Regulares Importam
- 02** Anatomia de uma Expressão Regular
- 03** Os Blocos de Construção
- 04** String e Scanner na Prática
- 05** Pattern e Matcher em Ação
- 06** Casamento, Agrupamento e Captura
- 07** Substituindo Texto com Back References
- 08** Estudo de Caso: Telefone e Tag HTML

CAPÍTULO 01

Por que Expressões Regulares Importam

O ponto cego mais comum de quem programa: usar regex sem saber que é regex

Você provavelmente já escreveu uma expressão regular sem perceber

Toda vez que você chama `texto.split(";")` ou `texto.replaceAll("[0-9]", "")`, já está escrevendo uma expressão regular — só que uma bem simples. Uma expressão regular é, no fundo, apenas uma string. A diferença é que alguns caracteres dentro dela ganham um significado especial (os chamados **metacaracteres**), e é esse significado que permite descrever um padrão de texto — "qualquer e-mail válido", por exemplo — em vez de precisar descrever um texto exato, caractere por caractere.

Isso não é exclusividade do Java. Editores de texto, IDEs, ferramentas de linha de comando e praticamente qualquer linguagem de programação implementam algum dialeto de expressões regulares. O vocabulário muda um pouco de uma ferramenta para outra, mas a lógica por trás é a mesma — o que você aprender aqui vai valer em qualquer contexto onde for preciso buscar, validar ou transformar texto.

Onde isso realmente economiza tempo

Existem tarefas que, sem regex, viram uma sequência cansativa de laços, condicionais e chamadas de índice manual dentro de uma String. Com uma expressão regular bem escrita, a mesma tarefa cabe em uma linha. Os casos mais comuns são:

- **Validação** — confirmar que uma entrada segue um formato esperado, como um e-mail ou um número de chamado.
- **Busca** — encontrar ocorrências de um padrão dentro de um texto maior, como vasculhar um arquivo de log em busca de um identificador específico.
- **Substituição** — trocar trechos que casam com um padrão por outro conteúdo.
- **Extração** — capturar só as partes relevantes de um texto, descartando o resto.
- **Divisão** — quebrar um texto em pedaços, usando o padrão como separador.

Quem já deu suporte a um sistema em produção conhece bem essa cena: abrir um arquivo de log enorme para rastrear o histórico de um chamado específico, ou o instante exato em que um erro aconteceu. Esse é exatamente o tipo de trabalho onde uma regex bem construída poupa horas de leitura manual — e é o cenário que esta apostila vai usar como fio condutor: uma central de atendimento técnico, com seus tickets, logs e formulários de cadastro.

◆ Nota

É normal estranhar a sintaxe de regex à primeira vista — ela realmente não lembra nada do resto do código Java. O investimento inicial em entender os metacaracteres se paga rápido, e reaparece o tempo todo em qualquer projeto que envolva texto, log ou validação de entrada.

CAPÍTULO 02

Anatomia de uma Expressão Regular

O que é, para que serve, e onde o Java expõe regex na sua API

Uma string com instruções embutidas

Por baixo do jargão, uma expressão regular continua sendo apenas uma string. A diferença é que ela pode conter combinações de caracteres com significado especial, interpretadas por um mecanismo chamado processador de expressões regulares. Esse processador lê o padrão, entende as instruções embutidas nele e usa essas instruções para decidir se um outro texto "casa" (dá match) com aquele padrão.

Ler uma expressão regular pronta não é trivial, e isso é esperado — a não ser que você as escreva com muita frequência, vai continuar consultando a documentação de tempos em tempos, e não há problema nenhum nisso. A boa notícia é que você raramente precisa inventar um padrão do zero: a maioria das validações comuns (e-mail, CEP, telefone) já foi escrita e discutida publicamente, e normalmente você encontra várias soluções diferentes para a mesma tarefa. Como em qualquer código, raramente existe uma única resposta certa — vale a pena entender bem o requisito antes de escolher qual adaptar.

As razões práticas para usar regex

Em vez de escrever manualmente a lógica de percorrer caractere por caractere, comparar posições e controlar índices, você descreve *o que* quer encontrar e deixa o motor de regex decidir *como* encontrar. Isso troca dezenas de linhas de código imperativo por uma linha declarativa — usada para confirmar que uma entrada está formatada corretamente, localizar ocorrências de um padrão, substituir trechos, extrair partes relevantes ou dividir um texto por um separador.

Regex espalhada pela API do Java

Várias classes do Java aceitam uma expressão regular como parâmetro direto — `String`, `Scanner`, `Formatter`, `DateTimeFormatter` e `Duration` são exemplos. É comum nem perceber que está usando regex: métodos como `String.format` e `printf` recorrem a padrões internos para interpretar especificadores como `%s` ou `%d`. Quando você precisa de mais controle — reaproveitar o mesmo padrão em várias buscas, capturar grupos, navegar de match em match — o Java concentra essa lógica no pacote `java.util.regex`, principalmente nas classes `Pattern` e `Matcher`, que são o assunto central do restante desta apostila.

Retorno	Método de <code>String</code> que aceita regex
boolean	<code>matches(String regex)</code>
String	<code>replaceAll(String regex, String replacement)</code>
String	<code>replaceFirst(String regex, String replacement)</code>
String[]	<code>split(String regex)</code>
String[]	<code>split(String regex, int limit)</code>

■ Dica

Você já usa `split`, `replaceAll` e companhia com strings literais simples, sem perceber que eles aceitam padrões muito mais ricos. O poder de verdade aparece quando o parâmetro deixa de ser um texto fixo e passa a ser uma expressão regular — é isso que o próximo capítulo começa a explorar.

CAPÍTULO 03

Os Blocos de Construção

Literais, classes de caracteres, quantificadores, âncoras e grupos

Cinco peças, infinitas combinações

Por mais complexa que uma expressão regular pareça, ela é sempre montada combinando apenas cinco tipos de peça. Depois de reconhecer cada uma isoladamente, ler qualquer regex vira um exercício de identificar essas peças em sequência.

- **Literais** — o caso mais simples: um caractere (ou sequência deles) que precisa casar exatamente, sem interpretação especial. O literal "TICKET" só casa com essa sequência exata de letras.
- **Classes de caracteres** — um jeito de dizer "qualquer um destes caracteres, aqui". Algumas já vêm prontas (o ponto, por exemplo, representa "qualquer caractere"); outras você define colocando os caracteres desejados entre colchetes.
- **Quantificadores** — controlam quantas vezes a peça anterior pode se repetir para ainda contar como match.
- **Âncoras (boundary matchers)** — não casam com um caractere, mas com uma posição no texto, como o começo ou o fim da string.
- **Grupos** — parênteses que isolam uma subexpressão, permitindo tratá-la como unidade e, mais importante, recuperar depois o texto exato que casou com ela.

Tipo	Exemplos
Classes de caracteres	<code>.</code> <code>[abc]</code> <code>[a-g]</code> <code>[A-Z]</code> <code>[0-9]</code> <code>^[abc]</code> <code>\d</code> <code>\s</code> <code>\w</code>
Quantificadores	<code>*</code> <code>+</code> <code>?</code> <code>{n}</code> <code>{n,}</code> <code>{n,m}</code>
Âncoras	<code>^</code> <code>\$</code> <code>\b</code>
Grupos	<code>()</code>

Classes de caracteres: o operador "ou" implícito

Colocar caracteres entre colchetes cria uma classe: em vez de exigir todos os caracteres em sequência, o padrão passa a significar "uma ocorrência de qualquer um destes". Por exemplo, ao validar o código de prioridade de um chamado, o padrão `[PMB]` casa com um único caractere que seja P, M ou B — as iniciais de Prioridade Alta, Média ou Baixa usadas nos formulários da central de atendimento. O mesmo resultado pode ser escrito com o metacaractere de "ou", a barra vertical: `(P|M|B)` — a diferença é que colchetes descrevem um único caractere, enquanto o pipe dentro de um grupo pode alternar entre sequências inteiras.

Dentro de colchetes, o significado dos caracteres muda. Um ponto fora de colchetes é o metacaractere "qualquer caractere"; o mesmo ponto dentro de colchetes vira um ponto literal, sem poder especial nenhum. Na prática, dentro dos colchetes praticamente qualquer caractere é tratado como literal — as exceções que continuam com significado especial ali dentro são `^`, `-`, `]` e `\`.

Classes predefinidas e seus equivalentes POSIX

Boa parte dos intervalos usados no dia a dia já tem um atalho pronto na própria API do Java, o que evita reescrever `[0-9]` toda vez que você precisa de um dígito.

Intervalo	Classe predefinida	POSIX (US-ASCII)
Dígitos	<code>[0-9]</code>	<code>\d</code> ou <code>\p{Digit}</code>
Minúsculas	<code>[a-z]</code>	<code>\p{Lower}</code>
Maiúsculas	<code>[A-Z]</code>	<code>\p{Upper}</code>
Letra (maiús. + minús.)	<code>[a-zA-Z]</code>	<code>\p{Alpha}</code>
Palavra (letras+dígitos+_)	<code>[a-zA-Z_0-9]</code>	<code>\w</code> ou <code>\p{Alnum}</code> *
Espaços em branco	<code>[\t\n\x0B\f\r]</code>	<code>\s</code> ou <code>\p{Space}</code>

■ Atenção

As classes POSIX cobrem apenas o alfabeto US-ASCII. Se o formulário de cadastro da central precisa aceitar nomes com acentuação — Célia, José, Ítalo — prefira as classes de `java.lang.Character`, como `\p{javaLowerCase}`, que têm cobertura Unicode mais ampla. Além disso, `\p{Alnum}` não inclui o underscore, ao contrário de `\w`.

Quantificadores: quantas vezes é suficiente?

Cada quantificador responde a uma pergunta diferente sobre repetição. Memorizar esta tabela evita boa parte dos erros mais comuns ao escrever uma regex nova — trocar `+` por `*`, por exemplo, é um dos deslizes mais frequentes de quem está começando.

Quantificador	Pergunta que responde	Exemplo	Casa com
<code>*</code>	zero ou mais repetições?	<code>d*</code>	vazio, <code>d</code> , <code>dd</code> , <code>ddd</code>
<code>+</code>	uma ou mais repetições?	<code>d+</code>	<code>d</code> , <code>dd</code> , <code>ddd</code> (nunca vazio)
<code>?</code>	zero ou uma repetição?	<code>cor(es)?</code>	<code>cor</code> , <code>cores</code>
<code>{n}</code>	exatamente <code>n</code> repetições?	<code>d{4}</code>	<code>dddd</code>
<code>{n,}</code>	ao menos <code>n</code> repetições?	<code>d{2,}</code>	<code>dd</code> , <code>ddd</code> , <code>dddd...</code>
<code>{n,m}</code>	entre <code>n</code> e <code>m</code> repetições?	<code>d{3,4}</code>	<code>ddd</code> , <code>dddd</code>

Âncoras: marcando uma posição, não um caractere

As três âncoras mais usadas não consomem nenhum caractere da string — elas apenas checam onde o cursor de leitura está no momento do match.

Âncora	O que verifica	Observação
<code>^</code>	início da string	casa com uma posição antes do primeiro caractere
<code>\$</code>	fim da string	casa com uma posição depois do último caractere
<code>\b</code>	limite entre palavra e não-palavra	útil para não casar um trecho no meio de outra palavra

♦ Nota

Sempre que uma regex nova não estiver casando como você espera, comece revisando os quantificadores e as âncoras — são de longe os pontos onde pequenos deslizes (um `*` onde deveria ter um `+`, ou um `\b` esquecido) mais causam resultado inesperado.

CAPÍTULO 04

String e Scanner na Prática

Como usar regex sem nem precisar tocar em Pattern ou Matcher

Regex escondida nos métodos que você já usa

Antes de chegar às classes especializadas do pacote `java.util.regex`, vale explorar o quanto já dá para fazer só com os métodos de `String` e `Scanner`. Imagine um trecho de log de uma central de atendimento, com uma linha por chamado registrado no turno. Contar quantas linhas existem, contar palavras, ou trocar um marcador de status por outro são tarefas do dia a dia de quem processa esse tipo de arquivo.

ContadorDeLog.java

```
String logDoTurno =
"TICKET#4021 aberto por camila.reis - prioridade alta\n" +
"TICKET#4022 aberto por diego.matos - prioridade baixa\n" +
"TICKET#4023 aberto por camila.reis - prioridade media\n" +
"TICKET#4024 aberto por bruna.melo - prioridade alta";

// split aceita regex: \R casa com qualquer sequência de quebra de linha
String[] linhas = logDoTurno.split("\\R");
System.out.println(linhas.length); // 4

// \\s+ casa com um ou mais espaços em branco seguidos
String[] palavras = logDoTurno.split("\\s+");
System.out.println(palavras.length); // conta todas as palavras do log
```

No primeiro caso, usar o literal `"\n"` já resolveria para um log gerado no próprio sistema, mas `\R` é mais seguro: é o combinador de quebra de linha do Java, e casa com qualquer sequência Unicode de fim de linha — importante se o arquivo de log vier de outro sistema operacional. No segundo caso, `\s` é a classe de espaço em branco, e o `+` garante que múltiplos espaços seguidos contem como um único separador.

Substituindo com `replaceAll`

Suponha que a política da central mude, e todo chamado com prioridade alta precise ser destacado no log antes de ser arquivado. Um padrão simples resolve isso sem nenhum laço manual:

DestaqueDeUrgencia.java

```
String destacado = logDoTurno.replaceAll(
"prioridade alta", "prioridade [URGENTE]"
);
System.out.println(destacado);

// cada ocorrência exata de "prioridade alta" é substituída
```


Esse exemplo ainda usa um literal como padrão — funciona, mas não aproveita nada de regex de verdade. O ganho aparece quando o padrão passa a descrever uma *categoria* de texto em vez de um texto fixo, como qualquer sequência que termine em um sufixo específico, comece com uma letra maiúscula, ou siga um formato de código.

Scanner: percorrendo texto token por token

A classe Scanner também aceita expressões regulares, e é útil quando você precisa percorrer um texto pedaço por pedaço em vez de processá-lo de uma vez só. Por padrão, um Scanner usa espaço em branco como delimitador entre tokens — ou seja, cada chamada a `next()` devolve uma palavra.

PercorrendoPalavras.java

```
Scanner scanner = new Scanner(logDoTurno);
while (scanner.hasNext()) {
    System.out.println(scanner.next());
}
scanner.close();
// imprime cada palavra do log, uma por linha
```

O delimitador pode ser trocado por qualquer expressão regular através de `useDelimiter`. Trocando o delimitador padrão por `\R`, cada token passa a ser uma linha inteira do log em vez de uma palavra:

PercorrendoLinhas.java

```
Scanner porLinha = new Scanner(logDoTurno);
porLinha.useDelimiter("\R");
porLinha.tokens().forEach(System.out::println);
porLinha.close();
// cada elemento do fluxo agora é uma linha completa do log
```

O método `tokens()`, disponível desde o JDK 9, devolve um `Stream<String>` com cada token delimitado — uma alternativa mais direta ao laço `while (hasNext())` quando o objetivo é só percorrer tudo uma vez.

Combinando Scanner com Stream para contar por linha

É possível ir além e contar quantas palavras cada linha do log tem, encadeando `split` dentro de um `map`:

ContagemPorPalavra.java

```
long totalDeChamadosDeAltaPrioridade = Arrays.stream(logDoTurno.split("\R"))
    .map(linha -> linha.replaceAll("\p{Punct}", "")) // remove pontuação da linha
    .flatMap(linha -> Arrays.stream(linha.split("\s+")))
    .filter(palavra -> palavra.matches("alta"))
    .count();
```

```
System.out.println(totalDeChamadosDeAltaPrioridade); // 2
```

Vale reparar em um detalhe: `matches` exige que a *string inteira* passada satisfaça o padrão, e não apenas um trecho dela — por isso o filtro funciona comparando palavra por palavra, depois de já ter separado o texto em tokens individuais com `split`. Se o padrão fosse aplicado à linha inteira, ele nunca casaria, porque a linha tem muito mais texto do que só a palavra "alta".

findInLine: capturando informação de uma linha específica

Quando o interesse é só a próxima linha do texto sendo lido — o cabeçalho de um arquivo, por exemplo — o método `findInLine` do `Scanner` procura a primeira ocorrência do padrão dentro da linha atual e devolve esse trecho, sem precisar processar o restante do arquivo.

PrimeiraOcorrencia.java

```
Scanner leitor = new Scanner(logDoTurno);  
String primeiraPrioridade = leitor.findInLine("alta|media|baixa");  
System.out.println(primeiraPrioridade); // "alta", encontrado na primeira linha  
leitor.close();
```

Esse método devolve `null` quando não encontra nenhuma correspondência antes do fim da linha atual — para continuar a busca na linha seguinte, é preciso chamar `nextLine()` explicitamente antes de tentar de novo.

■ Dica

Os métodos de `String` e `Scanner` resolvem boa parte do dia a dia. Mas quando o mesmo padrão precisa ser reaproveitado em muitas buscas, ou quando é preciso navegar `match a match` com controle fino, o pacote `java.util.regex` — com `Pattern` e `Matcher` — entrega esse controle. É o assunto do próximo capítulo.

CAPÍTULO 05

Pattern e Matcher em Ação

Compilação, reuso, thread-safety e a diferença entre greedy e reluctant

Uso pontual vs. padrão reutilizável

Antes de falar em compilar, vale ver o caminho mais curto: o método estático `Pattern.matches(regex, texto)` testa uma string contra um padrão em uma única chamada, sem que você precise criar nada explicitamente.

VerificacaoPontual.java

```
String status = "chamado encerrado.";
boolean valido = Pattern.matches("^[A-Z].*\\.\\$", status);
System.out.println(valido);
```

O problema é que, por trás dos panos, esse método compila a expressão regular inteira toda vez que é chamado. Para uma verificação isolada, isso não pesa. Mas se você fosse aplicar essa mesma regra a milhares de chamados vindos de um arquivo, recompilar o padrão a cada chamado seria um desperdício — e é exatamente esse desperdício que `Pattern.compile` evita.

O que acontece quando você "compila" uma regex

Quando você passa uma string de expressão regular ao método estático `Pattern.compile(...)`, o Java não guarda apenas o texto — ele processa essa string e devolve uma instância de `Pattern` pronta para uso repetido. Esse processamento, chamado de compilação, envolve verificar a sintaxe da expressão, montar uma representação interna dela e otimizar essa representação para tornar o casamento mais rápido em cada uso posterior.

O ganho prático é reuso: uma vez compilado, o mesmo `Pattern` pode gerar quantos `Matcher` forem necessários, sem pagar de novo o custo de analisar a sintaxe. Isso importa especialmente quando o mesmo padrão é aplicado repetidamente — por exemplo, validando o formato do e-mail em cada novo cadastro de cliente que chega no sistema da central de atendimento.

ValidadorDeCadastro.java

```
public class ValidadorDeCadastro {

    // compilado uma única vez, reaproveitado em toda validação
    private static final Pattern EMAIL_VALIDO =
        Pattern.compile("^[\\w.+-]+@[\\w-]+\\. [a-zA-Z]{2,}$");

    public static boolean emailEhValido(String email) {
        return EMAIL_VALIDO.matcher(email).matches();
    }
}
```

O que o Matcher oferece além do match completo

Enquanto `Pattern` representa a regra compilada, `Matcher` é o objeto que efetivamente aplica essa regra a um texto concreto. Além de verificar se o texto inteiro casa com o padrão, o `Matcher` também sabe fazer matching parcial (através de `lookingAt` e das versões sobrecarregadas de `find`), dá acesso aos grupos capturados — assunto do próximo capítulo — e pode ser reaproveitado para avaliar strings diferentes sem precisar recompilar o padrão.

O preço desse reuso: estado mutável

Essa flexibilidade tem uma contrapartida: um `Matcher` guarda estado interno, que muda conforme você chama seus métodos. Isso tem duas consequências práticas — a instância não é thread-safe (duas threads usando o mesmo `Matcher` ao mesmo tempo podem corromper esse estado), e às vezes é preciso reiniciar esse estado explicitamente antes de avaliar um novo texto do início.

■ Atenção

Estado mutável em `Matcher` costuma ser a explicação para bugs intermitentes em código concorrente. Se vários atendentes validam formulários ao mesmo tempo em threads diferentes, crie um `Matcher` por thread a partir do mesmo `Pattern` compartilhado — o `Pattern` em si é imutável e seguro para compartilhar entre threads.

Greedy vs. reluctant: quem para primeiro

Por padrão, os quantificadores em Java são gulosos (*greedy*): tentam casar com a maior quantidade possível de caracteres antes de recuar, se necessário. A expressão `.*`, por exemplo, tenta engolir o texto inteiro primeiro, e só devolve caracteres se isso for indispensável para o resto do padrão bater.

Colocar um `?` logo depois do quantificador inverte esse comportamento, tornando-o relutante (*reluctant*): `.*?` tenta casar com o menor trecho possível, parando assim que o restante da expressão já for satisfeito. A diferença fica óbvia em textos com mais de uma ocorrência do mesmo delimitador — um padrão guloso tende a atravessar ocorrências no meio do caminho, enquanto o relutante para na primeira oportunidade.

GreedyVsReluctant.java

```
String observacoes =
"obs=\"cliente aguardando retorno\" turno=\"tarde\" obs=\"reaberto pelo cliente\"";

// greedy: tenta casar o máximo possível
Pattern greedy = Pattern.compile("obs=\".*\"");
// resultado: casa desde o PRIMEIRO obs=" até a ÚLTIMA aspas do texto inteiro

// reluctant: tenta casar o mínimo possível
Pattern reluctant = Pattern.compile("obs=\".*?\"");
// resultado: casa cada obs="..." separadamente, parando na primeira aspas de fechamento
```

No exemplo, o padrão guloso enxerga a string inteira como um único bloco entre a primeira e a última aspas — incluindo o trecho de `turno="tarde"` no meio, o que não é o resultado desejado. O padrão relutante, por parar assim que encontra a primeira aspas de fechamento, isola corretamente cada campo `obs="..."` como uma correspondência separada.

Sinalizadores: correspondência sem diferenciar maiúsculas e minúsculas

Por padrão, uma regex em Java diferencia maiúsculas de minúsculas — "ALTA" e "alta" são strings diferentes para o processador de expressões regulares. Quando um formulário de abertura de chamado aceita a prioridade em qualquer capitalização, existem duas formas de tornar a comparação insensível a isso. A primeira é passar uma constante como segundo argumento de `Pattern.compile`, afetando o padrão inteiro:

FlagDeCompilacao.java

```
Pattern prioridade = Pattern.compile(
    "alta|media|baixa", Pattern.CASE_INSENSITIVE
);
System.out.println(prioridade.matcher("ALTA").matches()); // true
```

A segunda forma é o sinalizador embutido dentro da própria expressão, através do grupo especial `(?i)`. A diferença é o alcance: colocado no início do padrão, ele afeta tudo o que vem depois; colocado dentro de um grupo específico, afeta só aquele trecho — uma forma mais cirúrgica de aplicar a regra apenas onde ela é necessária.

FlagInline.java

```
Pattern prioridadeInline = Pattern.compile("(?i)alta|media|baixa");
System.out.println(prioridadeInline.matcher("Media").matches()); // true
```

■ Dica

Prefira `Pattern.CASE_INSENSITIVE` quando a regra vale para o padrão inteiro — é mais legível do que embutir `(?i)` no meio da expressão. Reserve o sinalizador inline para os casos em que só uma parte específica do padrão precisa ignorar a capitalização.

CAPÍTULO 06

Casamento, Agrupamento e Captura

matches, lookingAt, find e como extrair pedaços de um match

Quatro formas de perguntar "isso casa?"

O `Matcher` oferece quatro métodos para verificar um casamento, todos retornando um `boolean`. A diferença entre eles está em quanto da string precisa casar, e de onde a busca recomeça.

Método	Comportamento
<code>matches()</code>	Exige que a string inteira, do primeiro ao último caractere, satisfaça o padrão.
<code>lookingAt()</code>	Exige apenas que o início da string satisfaça o padrão; o restante pode sobrar sem problema.
<code>find()</code>	Procura a próxima ocorrência a partir de onde a busca anterior parou — útil para percorrer vários matches em sequência.
<code>find(int start)</code>	Variante que já reinicia a busca a partir do índice informado.

Um detalhe pouco intuitivo: dentro de um `matches()`, um quantificador relutante pode acabar se comportando como se fosse guloso — já que casar a string inteira é obrigatório de qualquer forma, não sobra "espaço" para ele realmente ser econômico. Esse efeito não acontece com `lookingAt()`, onde o comportamento relutante é respeitado normalmente.

`find()` em loop: percorrendo todas as ocorrências

Diferente de `matches()` e `lookingAt()`, que sempre recomeçam do índice zero, `find()` guarda onde a última busca parou e continua dali na próxima chamada. É esse comportamento que permite percorrer todas as ocorrências de um padrão em um laço `while`:

Percorrendo Chamados.java

```
String logDoDia = "TICKET#101 aberto; TICKET#102 aberto; TICKET#103 reaberto";

Matcher m = Pattern.compile("TICKET#\\d+").matcher(logDoDia);
while (m.find()) {
    System.out.println(m.group() + " encontrado entre " + m.start() + " e " + m.end());
}

// TICKET#101 encontrado entre 0 e 10
// TICKET#102 encontrado entre 12 e 22
// TICKET#103 encontrado entre 24 e 34
```

Os métodos `start()` e `end()` devolvem os índices onde o último match começou e terminou — úteis quando você precisa da posição exata, e não só do texto. Se for necessário recomeçar a busca do início da string, basta chamar `reset()` no `Matcher`; sem isso, `find()` sempre parte de onde a chamada anterior parou.

Grupos: isolando o que interessa

Um grupo é qualquer trecho da expressão regular envolto em parênteses. Sozinho, um par de parênteses já muda o comportamento da regex de duas formas: primeiro, deixa claro a que trecho um quantificador se aplica; segundo — e mais importante — faz o mecanismo de regex lembrar qual texto exatamente casou com aquele trecho.

Esse "lembrar" é a captura. Depois que um match acontece, você pode perguntar ao Matcher o que cada grupo capturou, usando `group(int)` ou `group(String)` para grupos nomeados. Isso é útil quando você não quer só saber se um texto casou, mas *qual parte dele* é relevante — por exemplo, separar o número do chamado do seu código de prioridade.

ExtraindoDadosDoChamado.java

```
String idDoChamado = "TICKET#4023-MEDIA";
Pattern padrao = Pattern.compile("TICKET#(\\d+)-(ALTA|MEDIA|BAIXA)");
Matcher m = padrao.matcher(idDoChamado);

if (m.matches()) {
    String numero = m.group(1); // "4023"
    String prioridade = m.group(2); // "MEDIA"
    System.out.println(numero + " / " + prioridade);
}
```

◆ Nota

O grupo 0 sempre existe implicitamente e representa o match inteiro, mesmo que sua expressão não declare nenhum parêntese. Os grupos que você declara com (começam a contar a partir de 1, numerados na ordem em que os parênteses de abertura aparecem, da esquerda para a direita. Também é possível nomear grupos em vez de depender de índices numéricos, o que deixa o código de captura mais legível quando há vários grupos em jogo — trocando `group(2)` por algo como `group("prioridade")`.

Nomeando grupos em vez de contar parênteses

Quando uma expressão tem vários grupos, lembrar qual índice corresponde a qual pedaço vira um exercício de contar parênteses de cabeça. O metacaractere `?<nome>`, logo depois do parêntese de abertura, resolve isso dando um nome ao grupo — que pode ser usado tanto em `group` quanto em `start` e `end`.

GruposNomeados.java

```
String idDoChamado = "TICKET#4023-MEDIA";
Pattern padrao =
    Pattern.compile("TICKET#(?<numero>\\d+)-( ?<prioridade>ALTA|MEDIA|BAIXA)");
Matcher m = padrao.matcher(idDoChamado);

if (m.matches()) {
    System.out.println(m.group("numero")); // "4023"
```

```
System.out.println(m.group("prioridade")); // "MEDIA"  
System.out.println(m.start("prioridade")); // índice onde a prioridade começa  
}
```

■ Atenção

Chamar `group(n)` com um índice que não existe na expressão — por exemplo, pedir o grupo 2 em um padrão que só declarou um par de parênteses — lança `IndexOutOfBoundsException` em tempo de execução, não um erro de compilação. Vale conferir quantos grupos o padrão realmente declara antes de indexar um número "no chute".

CAPÍTULO 07

Substituindo Texto com Back References

MatchResult, o método results() e como reaproveitar grupos capturados

MatchResult: a interface por trás do Matcher

A classe `Matcher` implementa a interface `MatchResult`, que define métodos como `start`, `end` e `group` — os mesmos que você já usa para inspecionar um match. Um método particularmente útil é `results()`, que devolve um `Stream` contendo um `MatchResult` para cada ocorrência do padrão na string — uma forma direta de processar todos os matches de uma vez, em vez de escrever manualmente um laço chamando `find()` repetidamente.

SomaDeTempoDeEspera.java

```
String logDeEspera =
    "chamado 4021 esperou 12min; chamado 4022 esperou 5min; chamado 4023 esperou 27min";

Pattern minutos = Pattern.compile("(\\d+)min");
Matcher m = minutos.matcher(logDeEspera);

int totalDeMinutos = m.results()
    .mapToInt(r -> Integer.parseInt(r.group(1)))
    .sum();

System.out.println(totalDeMinutos); // 44
```

Como `results()` devolve um `Stream` comum, ele encadeia normalmente com o restante da API de streams. Somar apenas o tempo de espera dos chamados de prioridade alta, por exemplo, é uma questão de acrescentar um `filter` antes do `mapToInt`:

FiltrandoPorPrioridade.java

```
String logComPrioridade =
    "chamado 4021 esperou 12min prioridade=alta; " +
    "chamado 4022 esperou 5min prioridade=baixa; " +
    "chamado 4023 esperou 27min prioridade=alta";

Pattern registro = Pattern.compile("(\\d+)min prioridade=(alta|media|baixa)");
Matcher m = registro.matcher(logComPrioridade);

int minutosDeAltaPrioridade = m.results()
    .filter(r -> r.group(2).equals("alta"))
    .mapToInt(r -> Integer.parseInt(r.group(1)))
    .sum();
```

```
System.out.println(minutosDeAltaPrioridade); // 39
```

Back references: reaproveitando o que já foi capturado

Uma back reference é uma forma de dizer "o texto aqui precisa ser igual ao que um grupo anterior já capturou" — sem repetir literalmente o padrão daquele grupo. Ela aparece em dois contextos diferentes, com sintaxes ligeiramente distintas.

Contexto	Sintaxe	Exemplo
Dentro da própria expressão regular	barra invertida + número do grupo	\1 refere-se ao texto do grupo 1
Dentro de uma string de substituição	cifrão + número do grupo	\$1 insere o texto do grupo 1 no resultado
Grupo nomeado, em qualquer contexto	nome do grupo no lugar do índice	\k<ano> na regex, ou \$ano na substituição

Um uso prático de back reference dentro da própria regex é detectar palavras repetidas por engano em um texto — um erro comum de digitação em descrições de chamado, como "o cliente cliente relatou instabilidade". O padrão abaixo captura uma palavra em um grupo e exige que a mesma palavra apareça logo em seguida:

DetectorDeRepeticao.java

```
String descricao = "o cliente cliente relatou instabilidade no sistema sistema";

Pattern palavraDuplicada = Pattern.compile("\\b(\\w+)\\s+\\1\\b");
Matcher m = palavraDuplicada.matcher(descricao);

while (m.find()) {
    System.out.println("palavra repetida: " + m.group(1));
}
// palavra repetida: cliente
// palavra repetida: sistema
```

Aqui, `(\\w+)` captura uma palavra qualquer no grupo 1, e `\\1` exige que o texto seguinte, depois de um espaço, seja idêntico ao que já foi capturado — sem precisar saber de antemão qual palavra é essa.

■ Atenção

Uma back reference só pode apontar para um grupo que já tenha sido fechado antes dela no texto da expressão — não existe como referenciar, dentro da própria regex, um grupo que ainda está sendo aberto mais à frente.

CAPÍTULO 08

Estudo de Caso: Telefone e Tag HTML

Duas expressões clássicas, explicadas token por token

Com todo o vocabulário dos capítulos anteriores em mãos, vale desmontar duas expressões regulares muito conhecidas e entender exatamente o papel de cada caractere nelas.

Padrão	Expressão regular	Exemplos de match
Telefone (EUA)	<code>\\([0-9]{3}\\) [0-9]{3}-[0-9]{4}</code>	(800) 123-4567
Tag HTML	<code><(\\w+)[^>]*>([^\v<>]*)(</\\1>)*</code>	<code><h1>Título</h1></code> , <code>
</code>

Número de telefone, peça por peça

```
\\([0-9]{3}\\) [0-9]{3}-[0-9]{4}
```

- `\\(` — parênteses são, por padrão, metacaracteres de agrupamento. Como aqui o objetivo é casar com um parêntese literal (o que aparece antes do DDD), é preciso escapá-lo com uma barra invertida.
- `[0-9]{3}` — uma classe de dígitos de 0 a 9, seguida do quantificador exato: exatamente três dígitos, nem mais nem menos.
- `\\)` — o parêntese de fechamento, escapado pelo mesmo motivo do de abertura, seguido de um espaço literal simples.
- `[0-9]{3}-[0-9]{4}` — outro bloco de três dígitos, um hífen (que dispensa escape em Java) e finalmente quatro dígitos.

◆ Nota

O hífen entre os dois blocos de dígitos não precisa de nenhum tratamento especial — ele só ganharia um significado diferente se estivesse dentro de colchetes, onde poderia indicar um intervalo em vez de um caractere literal.

Do rígido ao flexível: refinando o padrão

O padrão anterior é bastante restritivo: só casa quando o número vem exatamente como (800) 123-4567, com parênteses e um único espaço. Na prática, um formulário de cadastro recebe o mesmo número escrito de formas diferentes — sem parênteses, sem espaço, ou com hífen no lugar do espaço. Refinar o padrão passo a passo, em vez de tentar acertar tudo de uma vez, deixa cada ajuste mais fácil de entender.

```
RefinandoTelefone.java
```

```
String numeros =
"(800) 123-4567\n" +
"800 123-4567\n" +
"800-123-4567";

// v1: exige os parênteses – só casa a primeira linha
Pattern v1 = Pattern.compile("\\(?:[0-9]{3}\\)? [0-9]{3}-[0-9]{4}");

// v2: parênteses continuam opcionais (o ? depois de cada um já resolve isso),
// mas o espaço entre os blocos ainda é obrigatório e literal

// v3: troca o espaço fixo por uma classe que aceita parêntese de fechamento
// OU espaço em branco, cobrindo as três variações de uma vez
Pattern v3 = Pattern.compile("\\(?:[0-9]{3}[\\]\\s-]*[0-9]{3}-[0-9]{4}");

long totalDeMatches = v3.matcher(numeros).results().count();
System.out.println(totalDeMatches); // 3
```

A ideia central é: `\(?: e \)` tornam os parênteses opcionais (o quantificador `?` casa com zero ou uma ocorrência do parêntese escapado). Já a classe `[\\]\s-]*` aceita, em qualquer combinação, um parêntese de fechamento, espaço em branco ou hífen entre o primeiro bloco de três dígitos e o segundo — o que cobre as três variações de formatação sem precisar de três expressões diferentes.

■ Dica

Vale evitar misturar `[0-9]`, `\\d` e `\\p{Digit}` dentro da mesma expressão — os três significam a mesma coisa, mas usar formas diferentes lado a lado só torna o padrão mais difícil de ler. Escolha uma convenção e mantenha-a consistente dentro de cada expressão.

Tag HTML, peça por peça

```
<(\w+)[^>]*>([^\v</>]*)(</\1>)*
```

- `<` — o colchete angular de abertura. Ele só vira metacaractere em contextos de nomeação de grupo; aqui o processador reconhece o contexto e trata o caractere como literal, sem exigir escape.
- `(\w+)` — o primeiro grupo, indexado automaticamente como grupo 1. `\w` representa "caractere de palavra", e o `+` exige ao menos uma ocorrência — este grupo captura o nome da tag, como `h1` ou `br`.
- `[^>]*` — o circunflexo inicial inverte o sentido da classe: em vez de "casa com estes caracteres", ela passa a significar "casa com qualquer caractere, exceto estes". Aqui, qualquer coisa que não seja um colchete angular de fechamento é aceita, zero ou mais vezes — o que cobre tanto tags simples quanto tags com atributos.
- `>` — o colchete angular que fecha a tag de abertura.
- `([^\v</>]*)` — o segundo grupo, responsável por capturar o conteúdo entre a tag de abertura e a de fechamento, excluindo quebras de linha (vertical whitespace), os próprios colchetes angulares e a barra invertida.
- `(</\1>)*` — o grupo final, que só existe para permitir aplicar um quantificador ao conjunto inteiro. Dentro dele está a barra de fechamento e uma back reference ao primeiro grupo (`\1`), exigindo que o rótulo de fechamento seja idêntico ao de abertura. O `*` no final torna esse grupo opcional, cobrindo tags sem fechamento explícito, como `<br/>`.

■ Dica

Boa parte dos intervalos usados nesses dois padrões tem equivalentes mais expressivos definidos pela própria API do Java — `[0-9]` pode virar `\d`, por exemplo, sem mudança de comportamento. Vale ler o Capítulo 03 sempre que uma dessas trocas parecer útil.

Vale reaproveitar a combinação de `results()` com `filter`, vista no Capítulo 07, para extrair só o que interessa de um trecho de HTML — por exemplo, isolando apenas as tags de cabeçalho (`h1`, `h2`, `h3`) e ignorando parágrafos e quebras de linha:

FiltrandoCabeçalhos.java

```
String trechoHtml = "<h1>Título</h1><p>Texto</p><h2>Subtítulo</h2><br/>";
Pattern tag = Pattern.compile("<(\w+)[^>]*>([^\v</>]*)(</\1>)*");

tag.matcher(trechoHtml).results()
    .filter(r -> r.group(1).toLowerCase().startsWith("h") && r.group(1).length() == 2)
    .forEach(r -> System.out.println(r.group(1) + ": " + r.group(2)));
// h1: Título
// h2: Subtítulo
```

Fim da apostila

Com literais, classes de caracteres, quantificadores, âncoras, grupos, Pattern, Matcher e back references no seu vocabulário, você já tem o suficiente para ler qualquer expressão regular que encontrar pela frente — e para escrever as suas próprias com confiança, em vez de colar a primeira que aparecer numa busca.